

Operaciones masivas y sincronización de relaciones

Por qué una FK cambia, pero la navegación todavía devuelve el valor anterior

La pregunta real: si `PlanchaAcero.OrdenId` y `PlanchaAcero.Orden` expresan la misma relación, ¿por qué, dentro del mismo `DbContext`, pueden devolver valores diferentes justo después de modificar uno de ellos? ¿Es normal? ¿Cómo se resuelve?

Respuesta central: con el seguimiento por instantáneas habitual, la asignación C# cambia inmediatamente la propiedad que escribes, pero no ejecuta automáticamente el mecanismo que sincroniza la otra representación de la relación. EF Core lo hace cuando detecta los cambios: `DetectChanges()` o métodos que lo desencadenan, como `SaveChanges()`.

El documento incluye:

PARTE	CONTENIDO
1. Operaciones masivas	Batching de <code>SaveChanges</code> , <code>ExecuteUpdate</code> , <code>ExecuteDelete</code> , <code>SqlBulkCopy</code> , transacciones y criterios de elección.
2. Estados y relaciones	Análisis del ejemplo de consola, causa interna del fenómeno, pruebas ejecutadas, soluciones y ejemplos análogos.
Anexos	Matriz de diagnóstico, laboratorio reproducible, enlaces a Microsoft Learn y GitHub Actions.

Ejemplo: `AceriaData.Console/Program.cs` · `PlanchaAcero / OrdenFabricacion`. Prueba controlada con EF Core 8.

Validación: GitHub Actions; no equivale a ejecutar la aplicación completa en SQL Server LocalDB.

Edición ampliada · 8 de octubre de 2026

PARTE 1 · INSERT, UPDATE y DELETE por lotes

1.1. Tres técnicas que no debemos confundir

Cuando hablamos de «operaciones por lotes» se mezclan tres comportamientos distintos:

- **Batching:** varias sentencias SQL se envían conjuntamente en menos viajes a la base de datos, aunque cada entidad pueda tener su propio UPDATE o DELETE.
- **Operación basada en conjuntos:** una única sentencia UPDATE o DELETE cambia muchas filas que cumplen un filtro, sin crear objetos EF por cada fila.
- **Bulk / carga masiva:** APIs especializadas transfieren o fusionan grandes volúmenes de registros, por ejemplo SqlBulkCopy y tablas de staging.

API / TÉCNICA	INSERT	UPDATE	DELETE	OBSERVACIONES
SaveChanges + batching	Sí	Sí	Sí	Mantiene seguimiento y grafos de entidades.
ExecuteUpdate	No	Sí	No	Una operación SQL por invocación; evita ChangeTracker.
ExecuteDelete	No	No	Sí	Borrado por condición, SQL inmediato.
SqlBulkCopy	Sí	No directo	No directo	Inserciones masivas en SQL Server.
Staging / Bulk externo	Sí	Sí	Sí	Evaluar API, transacción y licencias.

1.2. SaveChanges: agrupa comandos, pero no fusiona todas las filas

```
foreach (var pedido in pedidosRastreados)
    pedido.Estado = "Procesado";

await context.SaveChangesAsync();
```

EF Core 8 detecta los cambios pendientes y, según el proveedor, agrupa varios comandos por lote. En SQL Server el límite máximo predeterminado es de 42 sentencias por lote; es un parámetro ajustable, no una promesa de rendimiento. Seguirá generando normalmente actualizaciones individuales por entidad.

```
options.UseSqlServer(connectionString,
    sql => sql.MaxBatchSize(100)); // Medir antes de adoptar
```

Regla: 10.000 entidades modificadas no equivalen automáticamente a un solo UPDATE. «Un SaveChanges» define una unidad lógica de persistencia, no una única instrucción SQL.

1.3. ExecuteUpdateAsync y ExecuteDeleteAsync

```
int actualizadas = await context.Pedidos
    .Where(p => p.Estado == "Pendiente")
    .ExecuteUpdateAsync(s => s
        .SetProperty(p => p.Estado, "Procesado"));

int eliminadas = await context.Pedidos
    .Where(p => p.Estado == "Cancelado")
    .ExecuteDeleteAsync();
```

La primera consulta genera conceptualmente UPDATE ... WHERE ... y la segunda DELETE ... WHERE Operan directamente contra las filas, sin cargar entidades y sin esperar a SaveChanges().

Precaución: no actualizan automáticamente los objetos que el DbContext ya esté rastreando; una entidad previamente cargada puede continuar con valores obsoletos en memoria.

1.4. INSERT masivo y cambios heterogéneos

AddRange + SaveChanges permite crear muchas entidades, pero EF Core 8 no dispone de un ExecuteInsert equivalente a ExecuteUpdate. Para cantidades muy grandes en SQL Server conviene evaluar SqlBulkCopy o una biblioteca Bulk. Si cada fila requiere un valor distinto, la combinación de staging + UPDATE con JOIN es una alternativa al UPDATE set-based uniforme.

```
// SQL Server, esquema conceptual:
// 1) SqlBulkCopy lleva (Id, NuevoEstado) a tabla staging.
// 2) SQL actualiza por join:
UPDATE p
SET p.Estado = s.NuevoEstado
FROM Pedidos AS p
JOIN StagingEstados AS s ON s.Id = p.Id;
```

1.5. Una Unidad de Trabajo puede contener varias operaciones masivas

```
await using var tx = await context.Database.BeginTransactionAsync();
try
{
    await context.Pedidos.Where(p => p.Estado == "Pendiente")
        .ExecuteUpdateAsync(s => s.SetProperty(p => p.Estado, "Procesado"));

    context.Auditorias.Add(new Auditoria { Accion = "Lote" });
    await context.SaveChangesAsync();
    await tx.CommitAsync();
}
catch { await tx.RollbackAsync(); throw; }
```

La transacción permite confirmar o revertir las operaciones como conjunto, pero no las convierte en una sola sentencia. Con SaveChanges, EF Core normalmente gestiona la transacción necesaria; con varias ExecuteUpdate/ExecuteDelete, si se desea atomicidad entre ellas hay que coordinar una transacción.

1.6. Selección práctica

NECESIDAD	HERRAMIENTA PREFERENTE
Cambios puntuales con entidades relacionadas	SaveChanges + ChangeTracker.
Miles de filas con el mismo criterio	ExecuteUpdate / ExecuteDelete.
Cientos de miles de INSERT en SQL Server	SqlBulkCopy / BulkInsert.
Muchas filas con nuevos valores individuales	Staging + JOIN / BulkUpdate.
Varios pasos que deben confirmarse juntos	Transacción explícita; controlar coherencia de entidades rastreadas.

Enlace con la parte 2: una operación set-based no pasa por el ChangeTracker. Por tanto, no sincroniza por sí sola las propiedades de navegación de objetos ya materializados. No mezcles sin revisar el mismo conjunto de filas con entidades rastreadas y ExecuteUpdate.

PARTE 2 · Por qué la FK cambia, pero la navegación aún no

2.1. Caso práctico: PlanchaAcero y OrdenFabricacion

En el ejemplo de consola, las dos órdenes se guardan primero y las tres planchas se agregan después al mismo contexto. El ejemplo original **no tenía DetectChanges()**: lo añadiste después de observar el valor antiguo.

```
// Fragmento del ejemplo de consola:
context.OrganizacionesFabricacion.AddRange(orden1, orden2);
context.SaveChanges(); // Las órdenes ya tienen Id real.

var plancha1 = new PlanchaAcero { OrdenId = orden1.Id, /* ... */ };
var plancha2 = new PlanchaAcero { OrdenId = orden1.Id, /* ... */ };
context.PlanchasAcero.AddRange(plancha1, plancha2);
// Ambas planchas siguen Added (no guardadas).
```

La relación está configurada de forma explícita:

```
b.HasOne(x => x.Orden)
  .WithMany(x => x.Planchas)
  .HasForeignKey(x => x.OrdenId)
  .OnDelete(DeleteBehavior.Cascade)
  .IsRequired();
```

La plancha es la entidad **dependiente** y la orden es la entidad **principal**. La propiedad OrdenId almacena la clave externa; Orden contiene una referencia a un objeto. Son **dos representaciones de la misma relación**, no dos alias de una misma variable.

2.2. Tu pregunta reducida a tres líneas

```
plancha2.OrdenId = orden2.Id;           // 1: asignas un entero nuevo
var valor = plancha2.OrdenId;           // 2: lees otra propiedad distinta
Console.WriteLine(valor);               // 3: aparece el Id anterior
```

Por qué tu expectativa era razonable: si EF Core sabe que OrdenId corresponde a Orden.Id y todo pertenece al mismo DbContext, parece natural esperar que modificar uno actualice inmediatamente el otro. **Pero DbContext no transforma los setters normales de C# en instrucciones reactivas que actualicen todo el grafo en ese mismo instante.**

2.3. Qué hace C# y qué hace EF Core: responsabilidades distintas

OPERACIÓN	QUIÉN LA REALIZA	EFFECTO INMEDIATO
plancha2.OrdenId = orden2.Id	Setter / CLR de C#	Cambia el número de OrdenId.
plancha2.Orden.Id	Getter / CLR de C#	Lee el objeto al que Orden todavía apunta.
ChangeTracker.DetectChanges()	Motor de seguimiento EF Core	Compara instantáneas y sincroniza FK y navegaciones.
SaveChanges()	EF Core + proveedor SQL	Detecta cambios por defecto y escribe en BD.

Cuando EF Core empieza a rastrear una entidad con estrategia de seguimiento **Snapshot**, guarda una instantánea de sus valores y relaciones conocidas. Una asignación normal de C# no tiene por qué notificarle nada en ese instante. Cuando se ejecuta la detección de cambios, EF Core compara lo que conocía con los valores actuales y reconoce: «La FK antes era 1 y ahora es 2». Entonces realiza el **Relationship Fixup**.

2.4. La secuencia exacta: antes de sincronizar / después de sincronizar

INSTANTE	OrdenId (FK)	Orden.Id (nav)	¿HAY UPDATE SQL?
Inicial	1	1	No
Tras OrdenId = 2	2	1	No
Tras DetectChanges()	2	2	No
Tras SaveChanges()	2	2	Sí, si procedía guardar

La discrepancia es **temporal** en la representación del grafo en memoria. No significa que EF Core haya rechazado el cambio: todavía no ha ejecutado el trabajo de sincronización.

Respuesta a la integridad referencial: no es un «bloqueo» de EF Core para proteger IDs. La integridad referencial se comprueba al persistir según las restricciones de la base de datos. Aquí ambas entidades y el nuevo Id pueden ser válidos; lo que falta antes de DetectChanges es el ajuste de referencias en memoria.

2.5. Los dos sentidos del problema en tu Program.cs

Caso A · Modificas la navegación y lees la FK

```
plancha1.Orden = orden2;           // C# cambia la referencia
Console.WriteLine(plancha1.OrdenId); // Puede mostrar Id anterior
context.ChangeTracker.DetectChanges();
Console.WriteLine(plancha1.OrdenId); // Ahora orden2.Id
```

EF Core compara la referencia actual `Orden` con la navegación registrada anteriormente. Detecta que la plancha ha pasado a pertenecer a `orden2` y actualiza `OrdenId` y las colecciones inversas rastreadas.

Caso B · Modificas la FK y lees la navegación

```
plancha2.OrdenId = orden2.Id;           // C# cambia el entero
Console.WriteLine(plancha2.OrdenId);    // Puede mostrar Id anterior
context.ChangeTracker.DetectChanges();
Console.WriteLine(plancha2.OrdenId);    // Ahora orden2.Id
```

EF Core detecta la variación de la FK y busca entre las entidades rastreadas una `OrdenFabricacion` con esa clave. Si está rastreada, corrige la referencia `Orden` y la colección `Planchas`. No necesita `SaveChanges` para hacer este ajuste.

2.6. Evidencia: la reproducción ejecutada en GitHub Actions

Se ejecutaron los dos sentidos dentro del mismo `DbContext` con EF Core 8 y SQLite en memoria; el objetivo fue aislar el mecanismo de `ChangeTracker`, no reproducir todos los aspectos de SQL Server LocalDB.

PRUEBA	JUSTO DESPUÉS DE ASIGNAR	DESPUÉS DE DetectChanges
FK → navegación	FK = 2; navegación.Id = 1	FK = 2; navegación.Id = 2
Navegación → FK	FK = 2; navegación.Id = 1	FK = 1; navegación.Id = 1

En ambas comprobaciones, las aserciones fueron **PASS**. La prueba separada que simulaba las planchas en estado `Added` también concluyó con éxito, antes y después de `SaveChanges`. No debemos interpretar estos resultados como ejecución integral de la aplicación con SQL Server LocalDB.

[GitHub Actions: experimento mínimo \(run 37807736877\)](#)

[Código reproducible: diagnostico/relaciones-minimas/Program.cs](#)

2.7. Por qué EF Core está diseñado así

El comportamiento no es casual: el seguimiento `Snapshot` permite usar **POCOs normales** (clases C# sin obligación de emitir eventos al cambiar una propiedad). El `DbContext` mantiene los valores originales y detecta diferencias en puntos apropiados: así desacopla el modelo de dominio de un seguimiento intrusivo en cada setter.

- **Ventaja:** código de entidades sencillo y sin mecanismos de notificación obligatorios.
- **Coste:** después de una asignación normal puede existir una ventana temporal en que FK y navegación no coinciden.
- **Compromiso:** EF Core ejecuta `DetectChanges` en puntos relevantes, especialmente `SaveChanges`; la aplicación puede solicitarlo expresamente si necesita coherencia inmediata del grafo.

Aunque EF Core rastree mil objetos, eso no significa que cada uno vigile todas las propiedades de los demás cada vez que cambian. La detección `Snapshot` compara valores y después aplica las reglas de `Relationship Fixup`.

2.8. Soluciones correctas: ¿cuál elegir?

SOLUCIÓN	CUÁNDO ES APROPIADA	QUÉ NO DEBES SUPONER
1 · Cambiar una sola navegación	Código orientado al modelo de objetos: <code>plancha.Orden = orden2</code> .	La FK no tiene por qué estar sincronizada al siguiente getter.
2 · Cambiar solo la FK	Conoces el Id destino y no necesitas inmediatamente el objeto destino.	La navegación puede continuar antigua o quedar null hasta detectarla/cargarla.
3 · DetectChanges explícito	Necesitas consultar ambos lados inmediatamente después de reasignar.	No ejecuta SQL ni reemplaza <code>SaveChanges</code> .
4 · SaveChanges al final	Quieres persistir al terminar la Unidad de Trabajo.	No es necesario solo para sincronizar memoria; puede ser costoso en cada paso.
5 · Notificaciones / proxies	Requisito real de detección inmediata o alto volumen tras medir.	Requieren configuración y requisitos de implementación.

Recomendación para tu caso: elige **una sola representación para escribir** (preferentemente la navegación cuando tienes la `OrdenFabricacion` ya rastreada), realiza los cambios de la Unidad de Trabajo y, **si vas a leer inmediatamente la otra representación para tomar decisiones**, usa `DetectChanges` explícito. Llama a `SaveChanges` cuando quieras persistir.

2.9. Atajos que generan confusión

- **Leer directamente una propiedad no detecta cambios:** `plancha.Orden.Id` usa el getter del objeto, no interroga automáticamente al `ChangeTracker`.
- **Algunas APIs sí disparan la detección:** `SaveChanges`, `ChangeTracker.Entries()`, `HasChanges()` y `DbSet.Local` en las condiciones predeterminadas.
- **DbContext.Entry(...)** detecta cambios localmente para esa entidad; en situaciones dependientes de modificaciones en otras entidades puede requerirse una detección global.
- **ChangeTracker.DebugView.LongView** no fuerza la detección: es útil para examinar la diferencia antes y después de `DetectChanges`.
- **AutoDetectChangesEnabled = false** evita detecciones automáticas; usarlo como «optimización» sin medir puede producir resultados incorrectos.

2.10. ¿Por qué no conviene poner SaveChanges entre cada línea?

Porque `SaveChanges` no es el método para resolver de forma local una navegación obsoleta: implica persistir a la base de datos, activar validaciones/restricciones y quizás abrir o participar en transacciones. Si solo quieres que el grafo de objetos refleje su nueva relación, `DetectChanges` es la operación pertinente.

```
// Correcto para inspeccionar en memoria (sin persistir):
plancha2.OrdenId = orden2.Id;
context.ChangeTracker.DetectChanges();
Console.WriteLine(plancha2.Orden.Id);

// Cuando hayas terminado la Unidad de Trabajo:
context.SaveChanges();
```

2.11. ¿Influye que la plancha todavía esté Added?

Aquí, las órdenes están guardadas y rastreadas; las planchas están en EntityState.Added. Eso no **impide** sincronizar FK y navegación. En nuestra primera prueba automática reproducimos precisamente este patrón y ambas direcciones funcionaron.

La llamada posterior `context.PlanchasAcero.Find(plancha1.Id)` es otra cuestión: antes del INSERT, una clave autonumérica de `PlanchaAcero` puede ser temporal dentro del `ChangeTracker` mientras la propiedad CLR todavía muestra 0. No utilices esa búsqueda como demostración de que FK y navegación se han sincronizado.

2.12. Tu entidad implementa INotifyPropertyChanging, pero no lo usa

```
public class PlanchaAcero : INotifyPropertyChanging
{
    public virtual int OrdenId { get; set; }
    public virtual OrdenFabricacion Orden { get; set; } = null!;
    public event PropertyChangingEventHandler? PropertyChanging;
}

// En DependencyInjection.cs está comentado:
// .UseChangeTrackingProxies()
```

La clase declara el evento, pero sus setters automáticos no lo emiten. Declarar una interfaz **no convierte mágicamente una clase en observable**. Además, no consta una estrategia de seguimiento basada en notificaciones configurada para este modelo; en las opciones de contexto, `UseChangeTrackingProxies` está comentado.

Por ello, la expectativa de notificación inmediata no corresponde a la configuración real mostrada. Si alguna vez se habilitan notificaciones, deben implementarse correctamente en todas las propiedades/navigaciones exigidas y configurarse la estrategia adecuada; los proxies de seguimiento son otra opción, con requisitos específicos. Ninguno es necesario para resolver este ejemplo.

PARTE 3 · Situaciones similares que también sorprenden

3.1. Cambiar la relación en la dirección inversa

Partimos de una orden A con una plancha, y una orden B sin ella. Modificamos la navegación de la plancha para apuntar a B:

```
plancha.Orden = ordenB;  
Console.WriteLine(plancha.OrdenId);      // Todavía puede indicar A  
context.ChangeTracker.DetectChanges();  
Console.WriteLine(plancha.OrdenId);      // Ahora indica B
```

Misma causa: el setter cambia una referencia C#; EF Core alinea la FK en la detección de cambios. No es que la base de datos impida cambiar la relación.

3.2. Agregar un hijo a una colección y leer su FK inmediatamente

```
ordenB.Planchas.Add(planchaNueva);  
Console.WriteLine(planchaNueva.OrdenId); // Puede no reflejar aún ordenB.Id  
context.ChangeTracker.DetectChanges();  
Console.WriteLine(planchaNueva.OrdenId); // Sincronizada
```

Si el cambio se hizo directamente sobre una colección con seguimiento Snapshot, la adición se reconoce al detectar cambios. EF Core ajusta la FK, la navegación inversa y, si corresponde, empieza a rastrear el hijo. Con entidades que emiten notificaciones bien configuradas el momento de detección puede cambiar.

3.3. Cambiar el estado de negocio no cambia por sí solo ninguna FK

```
plancha.Orden.Estado = "EnProduccion";  
Console.WriteLine(plancha.Orden.Estado); // "EnProduccion" inmediatamente  
// Ninguna de estas asignaciones cambia plancha.OrdenId.  
// La BD conserva el estado anterior hasta guardar.
```

Aquí NO hay Relationship Fixup pendiente para la propiedad Estado: es una propiedad normal, y leer la misma instancia devuelve al instante el valor nuevo. El seguimiento solo influye en cuándo se marca Modified y se persiste.

3.4. Misma Unidad de Trabajo, pero la consulta SQL lee el valor antiguo

```
plancha.Orden.Estado = "EnProduccion";  
var memoria = plancha.Orden.Estado; // "EnProduccion"  
var sql = context.OrdenesFabricacion  
    .Where(o => o.Id == plancha.OrdenId)  
    .Select(o => o.Estado).Single(); // Valor aún guardado en BD
```

Las proyecciones escalares se ejecutan en SQL Server sobre los datos persistidos. En cambio, una consulta con seguimiento que devuelve la entidad ya rastreada puede devolver la instancia que tienes en memoria. Mismo DbContext no implica que cada consulta se resuelva contra los cambios pendientes.

3.5. Cambiar a una orden que NO está siendo rastreada

```
plancha.OrdenId = idDeOrdenExistenteNoCargada;
context.ChangeTracker.DetectChanges();
Console.WriteLine(plancha.OrdenId); // Nuevo Id
Console.WriteLine(plancha.Orden?.Id); // Puede ser null
```

EF Core no materializa automáticamente una OrdenFabricacion solo porque se escriba su Id. Si el destino no está rastreado, la FK puede estar sincronizada como valor, pero la navegación no puede apuntar a un objeto que no existe en memoria. Se necesitará cargar la entidad si se requiere su objeto.

3.6. Romper una relación obligatoria y borrado en cascada

```
// Conceptual: quitar una plancha de una orden.
ordenA.Planchas.Remove(plancha);
context.ChangeTracker.DetectChanges();
// Si la relación es requerida y no se reasigna,
// EF Core puede marcar la dependiente como Deleted.
// La BD no se modifica hasta SaveChanges.
```

En tu configuración la relación es obligatoria y tiene DeleteBehavior.Cascade. Hay que distinguir **reasignación** a otra orden de **ruptura permanente** de una relación obligatoria: este segundo supuesto puede desencadenar comportamiento de huérfanos. No conviene extrapolar resultados de una relación opcional a una requerida.

3.7. Cambiar simultáneamente FK y navegación hacia destinos distintos

```
plancha.OrdenId = ordenB.Id;
plancha.Orden = ordenC; // Instrucciones contradictorias
context.ChangeTracker.DetectChanges();
```

Este código expresa dos intenciones incompatibles. El resultado del fixup puede depender de qué cambios se detecten y de su orden; no debe usarse como regla de precedencia de negocio. La documentación recomienda **modificar una única navegación o, si conviene, únicamente la FK**.

3.8. Desactivar detección automática de cambios

```
context.ChangeTracker.AutoDetectChangesEnabled = false;
plancha.OrdenId = ordenB.Id;
context.SaveChanges(); // NO presupongas detección automática

context.ChangeTracker.DetectChanges(); // Si quieres detectarlos
context.ChangeTracker.AutoDetectChangesEnabled = true;
```

Desactivar AutoDetectChangesEnabled cambia un supuesto central de todos los ejemplos. Es una optimización excepcional basada en mediciones y con restauración segura (try/finally). No se recomienda para solucionar una supuesta lentitud sin diagnóstico.

3.9. ¿Y si usamos proxies de seguimiento de cambios?

Los proxies pueden interceptar asignaciones sobre propiedades virtuales de instancias creadas por EF y notificar los cambios sin esperar a una exploración Snapshot; requieren paquete, configuración y modelo adecuados. Las entidades que implementan correctamente `INotifyPropertyChanged` / `INotifyPropertyChanging` y la estrategia configurada ofrecen otra vía.

Pero no recomendamos cambiar la arquitectura para este caso: basta con entender cuándo sincroniza EF Core, elegir una representación consistente para escribir y usar `DetectChanges` si necesitamos la otra inmediatamente.

PARTE 4 · Laboratorio y guía de diagnóstico

4.1. Experimento de tres líneas: reproducible y verificable

Se ejecuta con EF Core 8, SQLite en memoria, dos principales E1 (Id 1 y 2), un dependiente E2 y el mismo DbContext para todas las operaciones.

```
// E2 empieza con E1Id = 1; E2.E1 apunta a E1 con Id = 1.
e2.E1Id = e1B.Id;
Console.WriteLine($"FK={e2.E1Id}; Nav={e2.E1.Id}");
// Observado: FK=2; Nav=1

context.ChangeTracker.DetectChanges();
Console.WriteLine($"FK={e2.E1Id}; Nav={e2.E1.Id}");
// Observado: FK=2; Nav=2
```

La misma prueba, invirtiendo el sentido, comprueba que al cambiar la navegación E2.E1 hacia la entidad con Id = 1, la FK puede seguir temporalmente en 2 hasta detectar cambios.

4.2. Cómo inspeccionar el ChangeTracker sin ocultar el fenómeno

```
plancha2.OrdenId = orden2.Id;
Console.WriteLine(context.ChangeTracker.DebugView.LongView);
// Sin detección automática inducida por DebugView.
context.ChangeTracker.DetectChanges();
Console.WriteLine(context.ChangeTracker.DebugView.LongView);
```

Busca la plancha en la vista larga y compara OrdenId: ... FK con Orden: {Id: ...}. Antes de detectar el cambio pueden diferir; después deben alinearse, si el destino está rastreado.

4.3. Matriz rápida de diagnóstico

SÍNTOMA	EXPLICACIÓN HABITUAL	PRUEBA / SOLUCIÓN
FK nueva, navegación antigua	Asignación CLR con seguimiento Snapshot aún no detectada.	DetectChanges; inspeccionar DebugView.
Navegación nueva, FK antigua	Cambio de referencia pendiente de fixup.	DetectChanges; luego leer FK.
FK nueva, navegación null	Orden destino no rastreada o no cargada.	Cargar Orden destino si se necesita objeto.
Objeto muestra nuevo Estado, Select SQL muestra antiguo	Valor no persistido; proyección SQL lee la BD.	SaveChanges cuando corresponda.
Ambas siguen mal después de DetectChanges	Revisar instancia/contexto/modelo, tracking y escrituras incompatibles.	DebugView + Entry + configuración FK.
SaveChanges no detecta pendientes	AutoDetectChangesEnabled desactivado / mapeo o notificaciones.	Verificar opciones y estrategia de tracking.

4.4. Cuatro preguntas antes de buscar un supuesto fallo de EF Core

- **¿Qué modificaste?** ¿Un escalar Estado, una FK OrdenId o una navegación Orden?
- **¿Qué leíste?** ¿El mismo escalar, la propiedad opuesta de la relación, una consulta a entidad rastreada o una proyección SQL?
- **¿Se detectaron los cambios?** ¿Ejecutaste DetectChanges, Entries, HasChanges o SaveChanges, o estaba AutoDetectChangesEnabled desactivado?
- **¿Están rastreadas ambas entidades?** ¿El objeto destino realmente existe en el ChangeTracker y el modelo reconoce la FK?

4.5. Conclusión que debe quedar clara

«Misma Unidad de Trabajo» no significa «sincronización instantánea de todas las propiedades relacionadas».

Cuando cambias `plancha.OrderId`, el entero cambia en ese momento. `plancha.Order` es otra referencia de C#, que EF Core corrige al detectar el cambio mediante Relationship Fixup. Esto es **normal, documentado y esperado** con seguimiento Snapshot. No se debe a un bloqueo de integridad referencial; tampoco exige guardar la operación en SQL Server para ver ambos lados coherentes.

4.6. Referencias contrastables

[Microsoft Learn · Change Detection and Notifications](#)

[Microsoft Learn · Changing Foreign Keys and Navigations](#)

[Microsoft Learn · Change Tracker Debugging](#)

[Microsoft Learn · Efficient Updating](#)

[Microsoft Learn · ExecuteUpdate / ExecuteDelete](#)

[Microsoft Learn · Transactions](#)

[GitHub · Prueba original con planchas Added \(SUCCESS\)](#)

[GitHub · Reproducción mínima de tres casos \(SUCCESS\)](#)

Alcance: esta guía analiza el ejemplo de consola y dos verificaciones automáticas de reproducción controlada. No se ha ejecutado la aplicación completa con una instancia real de SQL Server LocalDB.